AIDL LAB ASSIGNMENT – 15

NAME – SUFIYA AFREEN

ENROLLMENT NO – 2403A51067

BATCH & BRANCH – CSE-03

DATE – 29-10-26

Lab/Exp 15: Attention Mechanisms in PyTorch

Kaggle Dataset Link:

1. Download and preprocess the Question Answer Pairs dataset.

2. Implement the required PyTorch architecture.

3. Train the model using appropriate hyperparameters.

4. Evaluate using suitable performance metrics.

5. Visualize predictions, loss curves, or generated outputs as applicable.

6. Compare results with a baseline model and summarize observations.

```python
# STEP 1: INSTALL REQUIRED LIBRARIES

import subprocess
import sys

subprocess.check_call([
    sys.executable, "-m", "pip", "install", "-q",
    "datasets", "scikit-learn", "matplotlib", "seaborn"
])

# STEP 2: IMPORT LIBRARIES

import re
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import torch
import torch.nn as nn

from collections import Counter
from datasets import load_dataset
from torch.utils.data import Dataset, DataLoader
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)

# STEP 3: SET RANDOM SEEDS AND DEVICE

SEED = 42

random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)

# ============================================================
# STEP 4: DOWNLOAD THE QUESTION PAIRS DATASET
# ============================================================

print("\nDownloading Quora Question Pairs dataset...")

dataset = load_dataset("nyu-mll/glue", "qqp")

# Use subsets to make the experiment practical in Google Colab.
# Increase these values for more comprehensive experiments.

TRAIN_SIZE = 12000
VAL_SIZE = 2000
TEST_SIZE = 2000

train_df = pd.DataFrame(dataset["train"].shuffle(seed=SEED).select(
    range(min(TRAIN_SIZE, len(dataset["train"])))
))

val_df = pd.DataFrame(dataset["validation"].shuffle(seed=SEED).select(
    range(min(VAL_SIZE, len(dataset["validation"])))
))
```

```python
# The official QQP test labels are not available for evaluation.
# Therefore, use a held-out subset of the labeled training split.
# This subset is disjoint from the selected training examples.

test_source = dataset["train"].shuffle(seed=SEED + 1)

test_df = pd.DataFrame(test_source.select(
    range(TRAIN_SIZE, min(TRAIN_SIZE + TEST_SIZE, len(test_source)))
))

# Keep only rows with valid labels and nonempty questions.

def clean_dataframe(df):
    df = df[["question1", "question2", "label"]].copy()
    df = df.dropna(subset=["question1", "question2", "label"])
    df["label"] = pd.to_numeric(df["label"], errors="coerce")
    df = df.dropna(subset=["label"])
    df["label"] = df["label"].astype(int)
    df = df[df["label"].isin([0, 1])]
    df["question1"] = df["question1"].astype(str)
    df["question2"] = df["question2"].astype(str)
    df = df[
        (df["question1"].str.strip() != "") &
        (df["question2"].str.strip() != "")
    ]
    return df.reset_index(drop=True)

train_df = clean_dataframe(train_df)
val_df = clean_dataframe(val_df)
test_df = clean_dataframe(test_df)

print("\nDataset loaded successfully!")
print("Training samples:", len(train_df))
print("Validation samples:", len(val_df))
print("Testing samples:", len(test_df))

print("\nSample records:")
print(train_df.head())

# ============================================================
# STEP 5: TEXT PREPROCESSING AND VOCABULARY CREATION
# ============================================================

PAD_TOKEN = "<PAD>"
UNK_TOKEN = "<UNK>"

MAX_VOCAB_SIZE = 15000
MAX_LEN = 30

def tokenize(text):
    text = str(text).lower()
    return re.findall(r"\b\w+\b", text)

counter = Counter()

for q1, q2 in zip(train_df["question1"], train_df["question2"]):
    counter.update(tokenize(q1))
    counter.update(tokenize(q2))

vocab = {
    PAD_TOKEN: 0,
    UNK_TOKEN: 1
}

for word, count in counter.most_common(MAX_VOCAB_SIZE - 2):
    vocab[word] = len(vocab)

print("\nVocabulary size:", len(vocab))

def encode_question(text):
    tokens = tokenize(text)
    ids = [vocab.get(token, vocab[UNK_TOKEN]) for token in tokens]
    ids = ids[:MAX_LEN]

    if len(ids) == 0:
        ids = [vocab[UNK_TOKEN]]

    ids += [vocab[PAD_TOKEN]] * (MAX_LEN - len(ids))
```

```
        ids += [vocab[PAD_TOKEN]] * (MAX_LEN - len(ids))
        return ids

    # ============================================================
    # STEP 6: CREATE PYTORCH DATASET AND DATALOADERS
    # ============================================================

    class QuestionPairDataset(Dataset):

        def __init__(self, dataframe):
            self.q1 = [
                encode_question(x) for x in dataframe["question1"]
            ]
            self.q2 = [
                encode_question(x) for x in dataframe["question2"]
            ]
            self.labels = dataframe["label"].tolist()

        def __len__(self):
            return len(self.labels)

        def __getitem__(self, index):
            return (
                torch.tensor(self.q1[index], dtype=torch.long),
                torch.tensor(self.q2[index], dtype=torch.long),
                torch.tensor(self.labels[index], dtype=torch.long)
            )

    BATCH_SIZE = 64

    train_loader = DataLoader(
        QuestionPairDataset(train_df),
        batch_size=BATCH_SIZE,
        shuffle=True
    )

    val_loader = DataLoader(
        QuestionPairDataset(val_df),
        batch_size=BATCH_SIZE,
        shuffle=False
    )

    test_loader = DataLoader(
        QuestionPairDataset(test_df),
        batch_size=BATCH_SIZE,
        shuffle=False
    )

    # ============================================================
    # STEP 7: DEFINE ATTENTION MECHANISM
    # ============================================================

    class Attention(nn.Module):

        def __init__(self, hidden_dim):
            super().__init__()

            self.attention_layer = nn.Linear(hidden_dim, 1)

        def forward(self, features, mask):
            # features shape: (batch, sequence_length, hidden_dim)

            scores = self.attention_layer(features).squeeze(-1)

            # Prevent padding tokens from receiving attention.
            scores = scores.masked_fill(~mask, -1e9)

            weights = torch.softmax(scores, dim=1)

            # Weighted sum of token representations.
            context = torch.sum(
                features * weights.unsqueeze(-1),
                dim=1
            )

            return context, weights
```

```python
# ============================================================
# STEP 8: DEFINE BASELINE AND ATTENTION MODELS
# ============================================================

class QuestionPairModel(nn.Module):

    def __init__(
        self,
        vocab_size,
        embedding_dim=64,
        hidden_dim=64,
        use_attention=False
    ):
        super().__init__()

        self.use_attention = use_attention
        self.embedding = nn.Embedding(
            vocab_size,
            embedding_dim,
            padding_idx=0
        )

        self.lstm = nn.LSTM(
            input_size=embedding_dim,
            hidden_size=hidden_dim,
            batch_first=True,
            bidirectional=True
        )

        output_dim = hidden_dim * 2

        if self.use_attention:
            self.attention = Attention(output_dim)

        # Combine both questions and their absolute difference.
        self.classifier = nn.Sequential(
            nn.Linear(output_dim * 3, 128),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(128, 2)
        )

    def encode(self, tokens):

        mask = tokens != 0

        embedded = self.embedding(tokens)

        outputs, _ = self.lstm(embedded)

        if self.use_attention:
            representation, weights = self.attention(outputs, mask)
        else:
            # Baseline: average valid LSTM outputs.
            mask_float = mask.unsqueeze(-1).float()

            representation = (
                (outputs * mask_float).sum(dim=1) /
                mask_float.sum(dim=1).clamp(min=1.0)
            )

            weights = None

        return representation, weights

    def forward(self, q1, q2, return_attention=False):

        rep1, weights1 = self.encode(q1)
        rep2, weights2 = self.encode(q2)

        difference = torch.abs(rep1 - rep2)

        combined = torch.cat(
            [rep1, rep2, difference],
            dim=1
        )

        logits = self.classifier(combined)
```

```
        if return_attention:
            return logits, weights1, weights2

        return logits

# ============================================================
# STEP 9: TRAINING FUNCTION
# ============================================================

EPOCHS = 5
LEARNING_RATE = 0.001

def train_model(model, train_loader, val_loader, model_name):

    model = model.to(device)

    criterion = nn.CrossEntropyLoss()

    optimizer = torch.optim.Adam(
        model.parameters(),
        lr=LEARNING_RATE
    )

    history = {
        "train_loss": [],
        "val_loss": [],
        "val_accuracy": []
    }

    print("\nTraining:", model_name)

    for epoch in range(EPOCHS):

        model.train()

        total_loss = 0.0

        for q1, q2, labels in train_loader:

            q1 = q1.to(device)
            q2 = q2.to(device)
            labels = labels.to(device)

            optimizer.zero_grad()

            outputs = model(q1, q2)

            loss = criterion(outputs, labels)

            loss.backward()

            torch.nn.utils.clip_grad_norm_(
                model.parameters(),
                max_norm=1.0
            )

            optimizer.step()

            total_loss += loss.item() * labels.size(0)

        train_loss = total_loss / len(train_loader.dataset)

        model.eval()

        val_total_loss = 0.0
        val_predictions = []
        val_targets = []

        with torch.no_grad():

            for q1, q2, labels in val_loader:

                q1 = q1.to(device)
                q2 = q2.to(device)
                labels = labels.to(device)

                outputs = model(q1, q2)
```

```
                loss = criterion(outputs, labels)

                val_total_loss += loss.item() * labels.size(0)

                predictions = outputs.argmax(dim=1)

                val_predictions.extend(
                    predictions.cpu().numpy()
                )

                val_targets.extend(
                    labels.cpu().numpy()
                )

        val_loss = val_total_loss / len(val_loader.dataset)

        val_accuracy = accuracy_score(
            val_targets,
            val_predictions
        )

        history["train_loss"].append(train_loss)
        history["val_loss"].append(val_loss)
        history["val_accuracy"].append(val_accuracy)

        print(
            f"Epoch [{epoch + 1}/{EPOCHS}] | "
            f"Train Loss: {train_loss:.4f} | "
            f"Validation Loss: {val_loss:.4f} | "
            f"Validation Accuracy: {val_accuracy:.4f}"
        )

    return model, history

# ============================================================
# STEP 10: TRAIN BASELINE MODEL
# ============================================================

baseline_model = QuestionPairModel(
    vocab_size=len(vocab),
    use_attention=False
)

baseline_model, baseline_history = train_model(
    baseline_model,
    train_loader,
    val_loader,
    "Baseline BiLSTM"
)

# ============================================================
# STEP 11: TRAIN ATTENTION MODEL
# ============================================================

attention_model = QuestionPairModel(
    vocab_size=len(vocab),
    use_attention=True
)

attention_model, attention_history = train_model(
    attention_model,
    train_loader,
    val_loader,
    "BiLSTM with Attention"
)

# ============================================================
# STEP 12: EVALUATE MODELS
# ============================================================

def evaluate_model(model, test_loader, model_name):

    model.eval()

    all_predictions = []
    all_targets = []

    with torch.no_grad():
```

```
        for q1, q2, labels in test_loader:

            q1 = q1.to(device)
            q2 = q2.to(device)

            outputs = model(q1, q2)

            predictions = outputs.argmax(dim=1)

            all_predictions.extend(
                predictions.cpu().numpy()
            )

            all_targets.extend(labels.numpy())

    accuracy = accuracy_score(all_targets, all_predictions)

    precision = precision_score(
        all_targets,
        all_predictions,
        zero_division=0
    )

    recall = recall_score(
        all_targets,
        all_predictions,
        zero_division=0
    )

    f1 = f1_score(
        all_targets,
        all_predictions,
        zero_division=0
    )

    print("\n", "=" * 55)
    print("EVALUATION:", model_name)
    print("=" * 55)

    print(f"Accuracy : {accuracy:.4f}")
    print(f"Precision: {precision:.4f}")
    print(f"Recall   : {recall:.4f}")
    print(f"F1-score : {f1:.4f}")

    print("\nClassification Report:")
    print(
        classification_report(
            all_targets,
            all_predictions,
            labels=[0, 1],
            target_names=[
                "Different Questions",
                "Duplicate Questions"
            ],
            zero_division=0
        )
    )

    return {
        "Model": model_name,
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1-score": f1
    }, all_targets, all_predictions

baseline_results, y_true, baseline_predictions = evaluate_model(
    baseline_model,
    test_loader,
    "Baseline BiLSTM"
)

attention_results, _, attention_predictions = evaluate_model(
    attention_model,
    test_loader,
    "BiLSTM with Attention"
)
```

```python
# ============================================================
# STEP 13: COMPARE BASELINE AND ATTENTION MODELS
# ============================================================

results_df = pd.DataFrame([
    baseline_results,
    attention_results
])

print("\nMODEL COMPARISON")
print(results_df.to_string(index=False))

# ============================================================
# STEP 14: VISUALIZE TRAINING LOSS
# ============================================================

plt.figure(figsize=(10, 5))

plt.plot(
    baseline_history["train_loss"],
    marker="o",
    label="Baseline Training Loss"
)

plt.plot(
    baseline_history["val_loss"],
    marker="o",
    label="Baseline Validation Loss"
)

plt.plot(
    attention_history["train_loss"],
    marker="s",
    label="Attention Training Loss"
)

plt.plot(
    attention_history["val_loss"],
    marker="s",
    label="Attention Validation Loss"
)

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training and Validation Loss Comparison")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# ============================================================
# STEP 15: VISUALIZE VALIDATION ACCURACY
# ============================================================

plt.figure(figsize=(8, 5))

plt.plot(
    range(1, EPOCHS + 1),
    baseline_history["val_accuracy"],
    marker="o",
    label="Baseline BiLSTM"
)

plt.plot(
    range(1, EPOCHS + 1),
    attention_history["val_accuracy"],
    marker="s",
    label="BiLSTM with Attention"
)

plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.title("Validation Accuracy Comparison")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

```python
# STEP 16: CONFUSION MATRIX FOR ATTENTION MODEL
# ============================================================

cm = confusion_matrix(
    y_true,
    attention_predictions,
    labels=[0, 1]
)

plt.figure(figsize=(6, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Different", "Duplicate"],
    yticklabels=["Different", "Duplicate"]
)

plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix - Attention Model")
plt.tight_layout()
plt.show()

# ============================================================
# STEP 17: DISPLAY SAMPLE PREDICTIONS
# ============================================================

print("\nSAMPLE QUESTION PAIR PREDICTIONS")
print("=" * 70)

for i in range(min(10, len(test_df))):

    row = test_df.iloc[i]

    actual = int(row["label"])
    predicted = int(attention_predictions[i])

    actual_text = (
        "Duplicate" if actual == 1 else "Different"
    )

    predicted_text = (
        "Duplicate" if predicted == 1 else "Different"
    )

    print(f"\nQuestion 1: {row['question1']}")
    print(f"Question 2: {row['question2']}")
    print(f"Actual     : {actual_text}")
    print(f"Predicted  : {predicted_text}")
    print("-" * 70)

# ============================================================
# STEP 18: VISUALIZE ATTENTION WEIGHTS
# ============================================================

def show_attention(question1, question2):

    attention_model.eval()

    tokens1 = tokenize(question1)[:MAX_LEN]
    tokens2 = tokenize(question2)[:MAX_LEN]

    ids1 = [vocab.get(w, vocab[UNK_TOKEN]) for w in tokens1]
    ids2 = [vocab.get(w, vocab[UNK_TOKEN]) for w in tokens2]

    if len(ids1) == 0:
        tokens1 = [UNK_TOKEN]
        ids1 = [vocab[UNK_TOKEN]]

    if len(ids2) == 0:
        tokens2 = [UNK_TOKEN]
        ids2 = [vocab[UNK_TOKEN]]

    ids1 += [0] * (MAX_LEN - len(ids1))
    ids2 += [0] * (MAX_LEN - len(ids2))
```

```
        q1_tensor = torch.tensor(
            [ids1],
            dtype=torch.long,
            device=device
        )

        q2_tensor = torch.tensor(
            [ids2],
            dtype=torch.long,
            device=device
        )

        with torch.no_grad():

            logits, weights1, weights2 = attention_model(
                q1_tensor,
                q2_tensor,
                return_attention=True
            )

            probabilities = torch.softmax(logits, dim=1)[0]

            predicted = int(torch.argmax(probabilities).item())

            w1 = weights1[0, :len(tokens1)].cpu().numpy()
            w2 = weights2[0, :len(tokens2)].cpu().numpy()

        print("\nQUESTION 1:", question1)
        print("QUESTION 2:", question2)

        print(
            "Prediction:",
            "Duplicate" if predicted == 1 else "Different"
        )

        print(
            f"Probability of different questions: {probabilities[0].item():.4f}"
        )

        print(
            f"Probability of duplicate questions: {probabilities[1].item():.4f}"
        )

        fig, axes = plt.subplots(1, 2, figsize=(14, 4))

        axes[0].bar(tokens1, w1)
        axes[0].set_title("Attention Weights - Question 1")
        axes[0].set_ylabel("Attention Weight")
        axes[0].tick_params(axis="x", rotation=45)

        axes[1].bar(tokens2, w2)
        axes[1].set_title("Attention Weights - Question 2")
        axes[1].set_ylabel("Attention Weight")
        axes[1].tick_params(axis="x", rotation=45)

        plt.tight_layout()
        plt.show()

    # Demonstration using example questions.

    show_attention(
        "How can I learn Python programming?",
        "What is the best way to learn Python?"
    )

    # ============================================================
    # STEP 19: SAVE TRAINED MODELS
    # ============================================================

    torch.save(
        baseline_model.state_dict(),
        "/content/baseline_bilstm.pth"
    )

    torch.save(
        attention_model.state_dict(),
        "/content/attention_bilstm.pth"
    )
```

```python
print("\nModels saved successfully!")
print("Baseline model: /content/baseline_bilstm.pth")
print("Attention model: /content/attention_bilstm.pth")

# ============================================================
# STEP 20: FINAL SUMMARY
# ============================================================

print("\n" + "=" * 60)
print("EXPERIMENT COMPLETED SUCCESSFULLY")
print("=" * 60)

print("\nBaseline BiLSTM Accuracy:",
      f"{baseline_results['Accuracy']:.4f}")

print("Attention BiLSTM Accuracy:",
      f"{attention_results['Accuracy']:.4f}")

difference = (
    attention_results["Accuracy"] -
    baseline_results["Accuracy"]
)

if difference > 0:
    print(
        f"\nAttention improved accuracy by "
        f"{difference * 100:.2f} percentage points."
    )
elif difference < 0:
    print(
        f"\nAttention accuracy was "
        f"{abs(difference) * 100:.2f} percentage points "
        "lower than the baseline."
    )
else:
    print("\nBoth models achieved the same accuracy.")

print("\nObservations:")
print("1. BiLSTM learns contextual representations of questions.")
print("2. Attention assigns different weights to question tokens.")
print("3. Evaluation metrics compare duplicate-question predictions.")
print("4. Loss curves show how the models learn over epochs.")
print("5. Attention does not guarantee better accuracy.")
print("\nExperiment finished.")
```

```
Device: cpu

Downloading Quora Question Pairs dataset...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
README.md: 100% 35.3k/35.3k [00:00<00:00, 2.42MB/s]
qqp/train-00000-of-00001.parquet: reconstructing file: 100% 33.6MB / 33.6MB, 3.29MB/s
qqp/train-00000-of-00001.parquet: downloading bytes: 33.0MB, 3.23MB/s
qqp/validation-00000-of-00001.parquet: reconstructing file: 100% 3.73MB / 3.73MB, 367kB/s
qqp/validation-00000-of-00001.parquet: downloading bytes: 3.61MB, 356kB/s
qqp/test-00000-of-00001.parquet: reconstructing file: 100% 36.7MB / 36.7MB, 3.60MB/s
qqp/test-00000-of-00001.parquet: downloading bytes: 36.1MB, 3.53MB/s
Generating train split: 100% 363846/363846 [00:00<00:00, 488159.37 examples/s]
Generating validation split: 100% 40430/40430 [00:00<00:00, 207637.29 examples/s]
Generating test split: 100% 390965/390965 [00:00<00:00, 510497.63 examples/s]

Dataset loaded successfully!
Training samples: 12000
Validation samples: 2000
Testing samples: 2000

Sample records:
                                          question1  \
0  What would Rhaegar and Jon have thought of eac...
1  What are the pros and cons of being a driver f...
2                        Who first started NGO?
3  Where can I get india vs spain davis cup 2016 ...
4  How can an unmarried couple book a hotel room ...

                                          question2  label
0  What is the amount of torque a 2000 Jeep Chero...      0
1  What are the pros and cons of being a driver f...      0
2                  What should I do to start an NGO?      0
3  How can I get tickets for the India Spain Davi...      1
4  What hotel in Khowai would be safe for unmarri...      0

Vocabulary size: 15000

Training: Baseline BiLSTM
Epoch [1/5] | Train Loss: 0.5976 | Validation Loss: 0.5339 | Validation Accuracy: 0.7245
Epoch [2/5] | Train Loss: 0.4898 | Validation Loss: 0.5254 | Validation Accuracy: 0.7520
Epoch [3/5] | Train Loss: 0.4047 | Validation Loss: 0.5254 | Validation Accuracy: 0.7630
Epoch [4/5] | Train Loss: 0.3197 | Validation Loss: 0.5895 | Validation Accuracy: 0.7605
Epoch [5/5] | Train Loss: 0.2355 | Validation Loss: 0.5977 | Validation Accuracy: 0.7600
```

```
Training: BiLSTM with Attention
Epoch [1/5] | Train Loss: 0.5980 | Validation Loss: 0.5245 | Validation Accuracy: 0.7280
Epoch [2/5] | Train Loss: 0.4849 | Validation Loss: 0.5075 | Validation Accuracy: 0.7495
Epoch [3/5] | Train Loss: 0.3945 | Validation Loss: 0.5275 | Validation Accuracy: 0.7580
Epoch [4/5] | Train Loss: 0.3069 | Validation Loss: 0.5423 | Validation Accuracy: 0.7640
Epoch [5/5] | Train Loss: 0.2176 | Validation Loss: 0.6481 | Validation Accuracy: 0.7550

 ======================================================
EVALUATION: Baseline BiLSTM
======================================================
Accuracy : 0.7650
Precision: 0.6797
Recall   : 0.6980
F1-score : 0.6887

Classification Report:
                    precision    recall  f1-score   support

Different Questions       0.82      0.80      0.81      1255
Duplicate Questions       0.68      0.70      0.69       745

           accuracy                           0.77      2000
          macro avg       0.75      0.75      0.75      2000
       weighted avg       0.77      0.77      0.77      2000


 ======================================================
EVALUATION: BiLSTM with Attention
======================================================
Accuracy : 0.7525
Precision: 0.6801
Recall   : 0.6336
F1-score : 0.6560

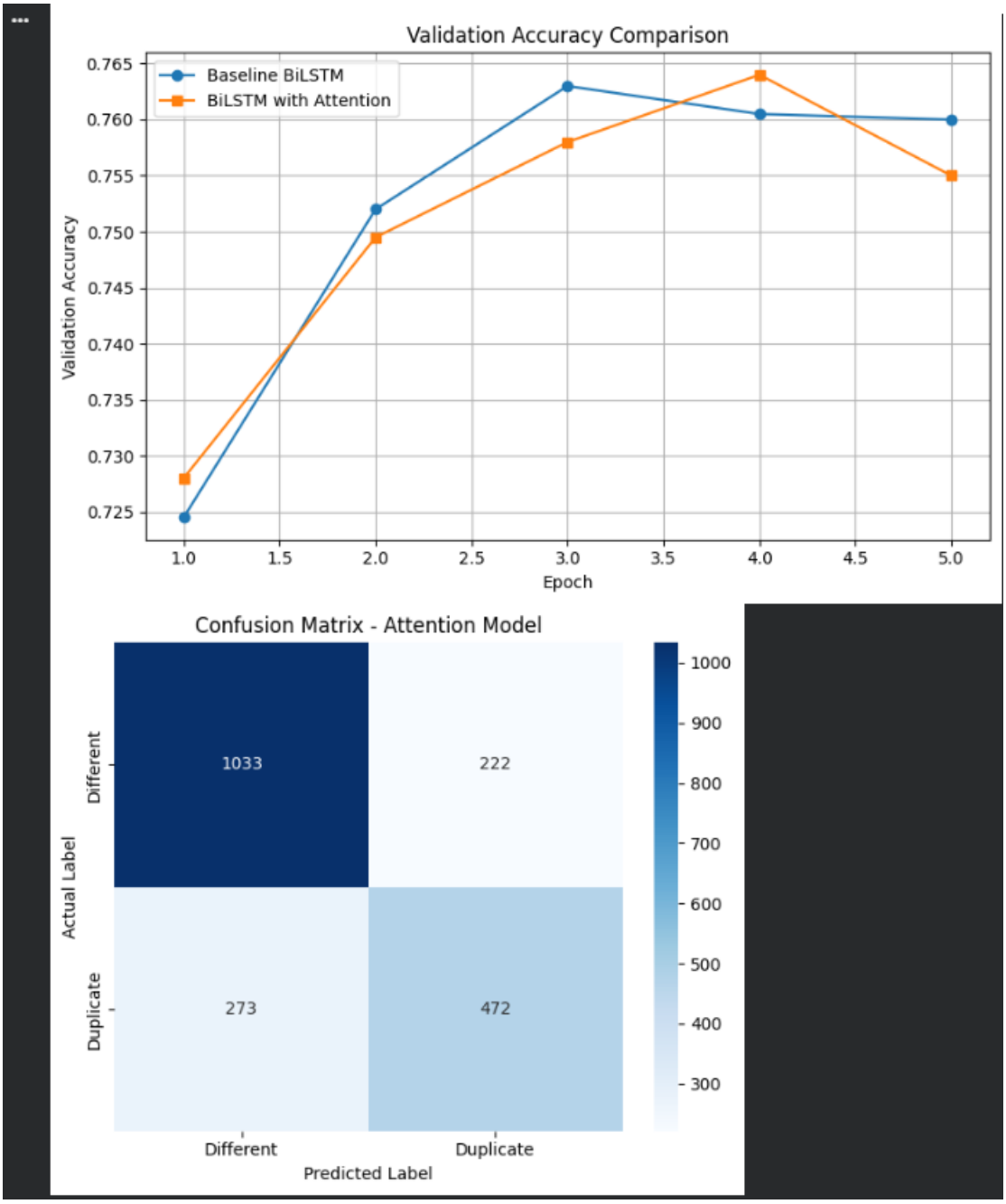
Classification Report:
                    precision    recall  f1-score   support

Different Questions       0.79      0.82      0.81      1255
Duplicate Questions       0.68      0.63      0.66       745

           accuracy                           0.75      2000
          macro avg       0.74      0.73      0.73      2000
       weighted avg       0.75      0.75      0.75      2000
```

```
                Model  Accuracy  Precision   Recall  F1-score
     Baseline BiLSTM    0.7650   0.679739 0.697987  0.688742
BiLSTM with Attention    0.7525   0.680115 0.633557  0.656011
```




Validation Accuracy Comparison
Baseline BiLSTM
BiLSTM with Attention
Validation Accuracy
Epoch
0.765
0.760
0.755
0.750
0.745
0.740
0.735
0.730
0.725
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5.0
Confusion Matrix - Attention Model
1033
222
273
472
Different
Duplicate
Actual Label
Predicted Label
1000
900
800
700
600
500
400
300

```
SAMPLE QUESTION PAIR PREDICTIONS
======================================================================

Question 1: Does Activia help lose weight?
Question 2: Why do some people think Activia Yogurt can help you lose weight?
Actual     : Duplicate
Predicted  : Different
----------------------------------------------------------------------

Question 1: How would Donald Trump's win affect India?
Question 2: How will Donald Trump getting elected as the President of the United States affect the relations of the USA and India?
Actual     : Duplicate
Predicted  : Different
----------------------------------------------------------------------

Question 1: Is it possible to initialize tensors in TensorFlow with numpy?
Question 2: Can I install TensorFlow in AWS?
Actual     : Different
Predicted  : Different
----------------------------------------------------------------------

Question 1: What are the best and worst things about public transit in Saint-Étienne, France? How could it be improved?
Question 2: What are the best and worst things about public transit in Bordeaux, France? How could it be improved?
Actual     : Different
Predicted  : Duplicate
----------------------------------------------------------------------

Question 1: Why I feel jealous of so many writers (girls) on Quora who have pretty faces and have huge followers even though their answers are not worthy enough?
Question 2: How does one get  followers on Quora without posting anything?
Actual     : Different
Predicted  : Different
----------------------------------------------------------------------

Question 1: Which is the best free blog platform?
Question 2: How do I start a blog and what are the best blogging platforms available?
Actual     : Duplicate
Predicted  : Different
----------------------------------------------------------------------

Question 1: How can I manage my anger?
Question 2: What should I do to overcome my anger?
Actual     : Duplicate
Predicted  : Duplicate
----------------------------------------------------------------------

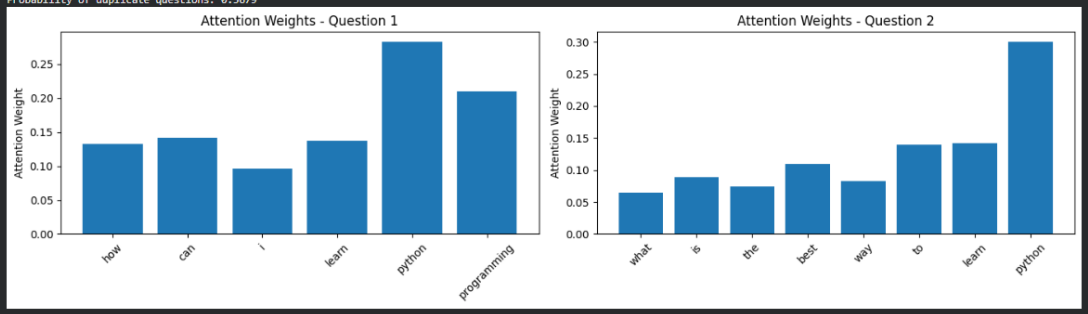
Question 1: How do I become journalist?
Question 2: What is the best way to become a journalist?
Actual     : Duplicate
Predicted  : Duplicate
----------------------------------------------------------------------

Question 1: What hotel in Nallamala Hills Hill-station would be safe for unmarried couples, without the harassment of police, hotel staff, and moral police?
Question 2: What hotel in Ayyampuzha Hill-station would be safe for unmarried couples, without the harassment of police, hotel staff, and moral police?
Actual     : Different
Predicted  : Different
```

```
----------------------------------------------------------------------

Question 1: What is the QuickBooks Hosting Support Number?
Question 2: What is Quickbooks Proadvisor?
Actual     : Duplicate
Predicted  : Different
----------------------------------------------------------------------

QUESTION 1: How can I learn Python programming?
QUESTION 2: What is the best way to learn Python?
Prediction: Different
Probability of different questions: 0.6321
Probability of duplicate questions: 0.3679
```




```
Models saved successfully!
Baseline model: /content/baseline_bilstm.pth
Attention model: /content/attention_bilstm.pth

============================================================
EXPERIMENT COMPLETED SUCCESSFULLY
============================================================

Baseline BiLSTM Accuracy: 0.7650
Attention BiLSTM Accuracy: 0.7525

Attention accuracy was 1.25 percentage points lower than the baseline.

Observations:
1. BiLSTM learns contextual representations of questions.
2. Attention assigns different weights to question tokens.
3. Evaluation metrics compare duplicate-question predictions.
4. Loss curves show how the models learn over epochs.
5. Attention does not guarantee better accuracy.

Experiment finished.
```